

ASSIGNMENT-13.5

NAME: SRAVANI

ROLLNO: 2303A510G7

BATCH: 30

TASK-1

PROMPT:

Refactor duplicate rectangle area and perimeter calculations into a reusable function with proper docstrings. Ensure output remains unchanged.

Legacy Code:

```
print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))

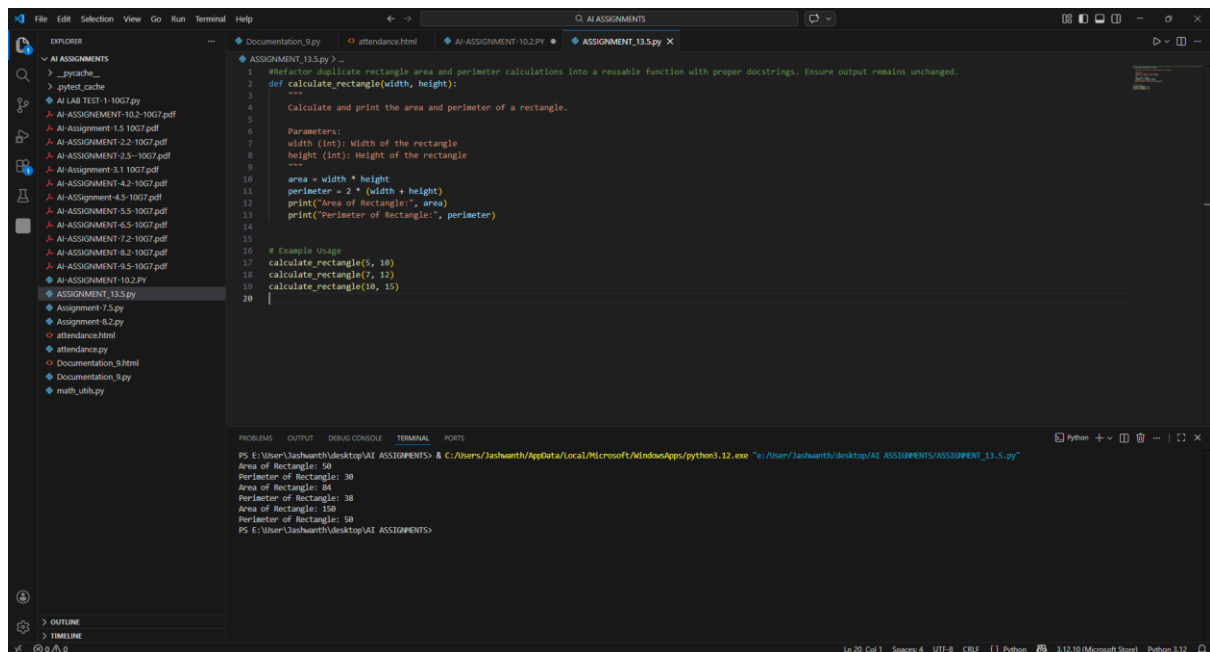
print("Area of Rectangle:", 7 * 12)

print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)

print("Perimeter of Rectangle:", 2 * (10 + 15))
```

CODE:



```
1  Refactor duplicate rectangle area and perimeter calculations into a reusable function with proper docstrings. Ensure output remains unchanged.
2  def calculate_rectangle(width, height):
3      """
4      Calculate and print the area and perimeter of a rectangle.
5
6      Parameters:
7      width (int): Width of the rectangle
8      height (int): Height of the rectangle
9      """
10     area = width * height
11     perimeter = 2 * (width + height)
12     print("Area of Rectangle:", area)
13     print("Perimeter of Rectangle:", perimeter)
14
15
16 # Example Usage
17 calculate_rectangle(5, 10)
18 calculate_rectangle(7, 12)
19 calculate_rectangle(10, 15)
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\User\Jashu\desktop\AI ASSIGNMENTS> & C:\Users\Jashu\AppData\Local\Microsoft\WindowsApps\python.12.exe "e:/User/Jashu/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 58
PS E:\User\Jashu\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

In this task, the rectangle area and perimeter formulas were repeated multiple times. After refactoring, we created one function to handle both calculations. This makes the code shorter and cleaner. If we ever need to change the formula, we only need to update it in one place instead of many places.

TASK-2

PROMPT:

Extract tax calculation into reusable function `calculate_total(price)` with docstrings.

Legacy Code:

```
price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

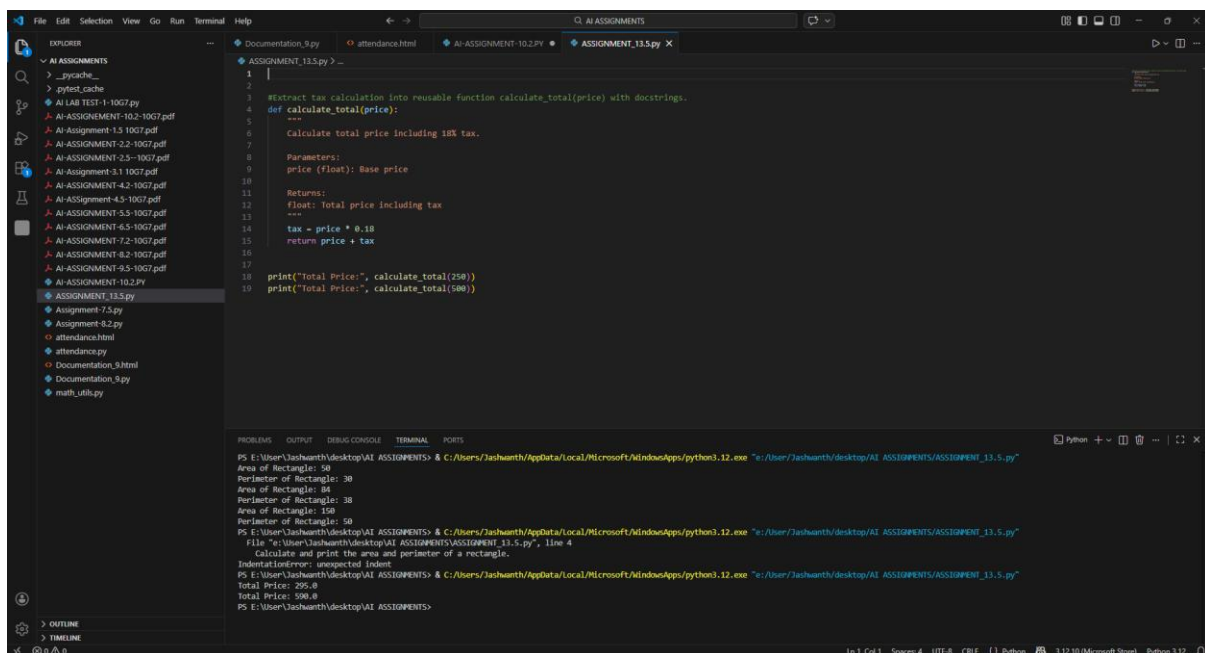
price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)
```

CODE:



```
1 |
2 |
3 | # Extract tax calculation into reusable function calculate_total(price) with docstrings.
4 | def calculate_total(price):
5 |     """
6 |     Calculate total price including 18% tax.
7 |
8 |     Parameters:
9 |     price (float): Base price
10 |
11 |     Returns:
12 |     float: Total price including tax
13 |     """
14 |     tax = price * 0.18
15 |     return price + tax
16 |
17 | print("Total Price:", calculate_total(250))
18 | print("Total Price:", calculate_total(500))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\Desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\Desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
File "e:\User\Jashwanth\Desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py", line 4
    Calculate and print the area and perimeter of a rectangle.
    ^
IndentationError: unexpected indent
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\Desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
Total Price: 295.0
Total Price: 598.0
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Earlier, the tax calculation was written directly in the main program. Now, it is placed inside a separate function. This makes the program more organized and reusable. We can use the same function for different prices without rewriting the logic again.

TASK-3

PROMPT:

Refactor grading logic into a class GradeCalculator with method calculate_grade(self, marks) using proper grading rules.

- The grading logic must strictly follow the conditions below:
 - Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
 - Marks $\geq 80 \rightarrow$ "Grade B"
 - Marks $\geq 70 \rightarrow$ "Grade C"
 - Marks $\geq 40 \rightarrow$ "Grade D"
 - Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

Legacy Code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

```
marks = 72
```

```
if marks >= 90:
```

```
    print("Grade A")
```

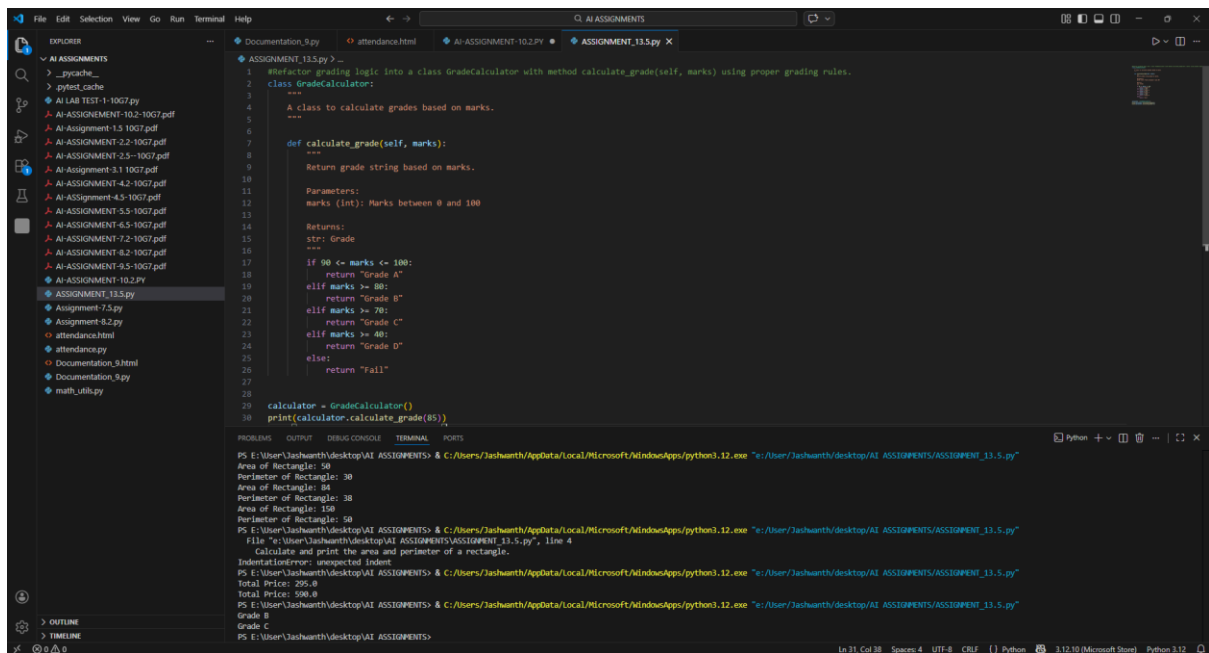
```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

CODE:



```
1 # Refactor grading logic into a class GradeCalculator with method calculate_grade(self, marks) using proper grading rules.
2 class GradeCalculator:
3     """
4     A class to calculate grades based on marks.
5     """
6
7     def calculate_grade(self, marks):
8         """
9         Return grade string based on marks.
10
11         Parameters:
12             marks (int): Marks between 0 and 100
13
14         Returns:
15             str: Grade
16         """
17         if 0 <= marks <= 100:
18             return "Grade A"
19         elif marks >= 80:
20             return "Grade B"
21         elif marks >= 70:
22             return "Grade C"
23         elif marks >= 40:
24             return "Grade D"
25         else:
26             return "Fail"
27
28 calculator = GradeCalculator()
29 print(calculator.calculate_grade(85))
```

```
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 64
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
File "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py", line 4
    Calculate and print the area and perimeter of a rectangle.
    ^
IndentationError: unexpected indent
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Total Price: 295.0
Total Price: 500.0
Grade B
Grade C
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Instead of writing grading conditions in the main program, we created a class for it. This follows Object-Oriented Programming (OOP). It makes the program more structured and easier to manage. If grading rules change in the future, we can update them easily inside the class.

TASK-4

PROMPT:

Separate input, processing, and output into modular functions.

Legacy Code:

```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

CODE:

```
1 #Separate input, processing, and output into modular functions.
2
3 def get_input():
4     """Get integer input from user."""
5     return int(input("Enter number: "))
6
7
8 def calculate_square(num):
9     """Return square of a number."""
10    return num * num
11
12
13 def display_result(result):
14     """Display the result."""
15     print("Square:", result)
16
17
18 number = get_input()
19 square = calculate_square(number)
20 display_result(square)
```

Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 58
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
File "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py", line 4
calculate and print the area and perimeter of a rectangle.
IndentationError: unexpected indent
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Total Price: 295.8
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Grade B
Grade C
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter number: 5
Square: 25
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>

OBSERVATION:

The program was divided into three parts: taking input, calculating the square, and displaying the result. This makes the code easy to understand and test. Each function has a specific role, which improves clarity.

TASK-5

PROMPT:

Convert salary calculation into EmployeeSalaryCalculator class using encapsulation.

Legacy Code:

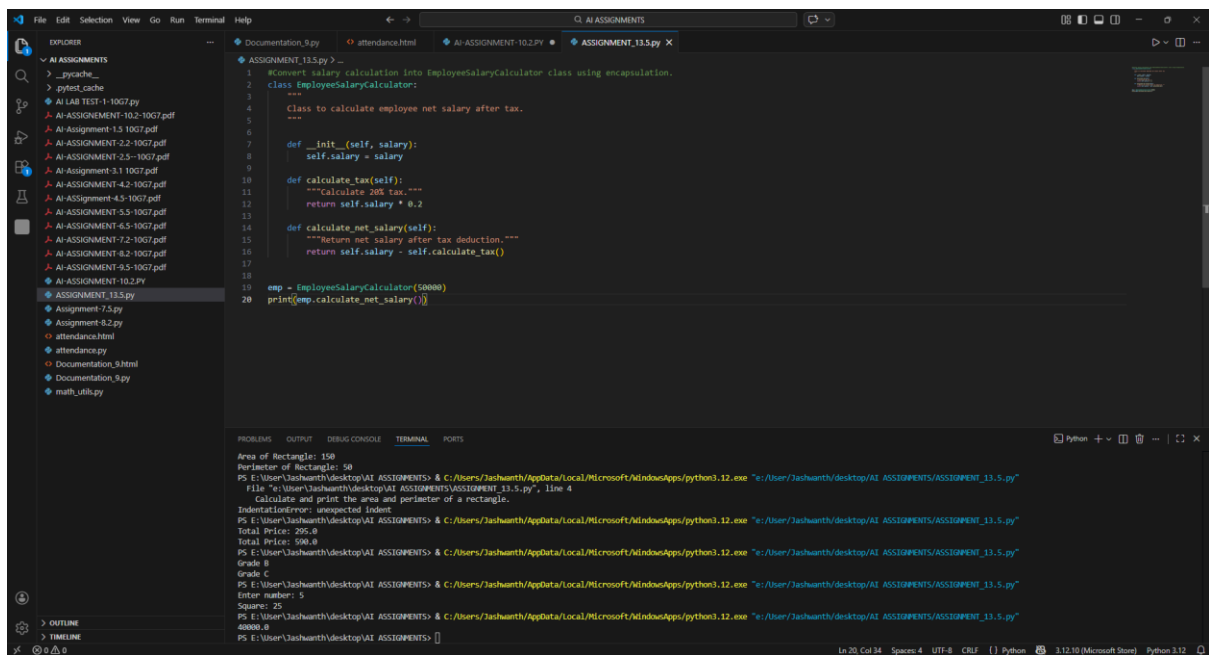
salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

CODE:



OBSERVATION:

Salary and tax calculations were organized inside a class. This keeps salary-related data and operations together. It also improves security and structure. The tax calculation is separated from net salary calculation, making the code clearer.

TASK-6

PROMPT:

Replace linear search with set lookup to improve time complexity.

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
if u == name:
```

```
found = True
```

```
print("Access Granted" if found else "Access Denied")
```

CODE:

The screenshot shows a VS Code editor with a file explorer on the left containing various assignment files. The main editor displays a Python script named `ASSIGNMENT_13.5.py` with the following code:

```
1 # Replace linear search with set lookup to improve time complexity.
2 users = {"admin", "guest", "editor", "viewer"}
3
4 name = input("Enter username: ")
5
6 if name in users:
7     print("Access Granted")
8 else:
9     print("Access Denied")
10
```

The terminal at the bottom shows the execution of the script. It displays the output of the `input` function and the resulting `print` statements. The output is as follows:

```
Total Price: 500.0
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Grade B
Grade C
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter number: 5
Square: 25
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
admin
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
File "e:/User/Jashwanth/Desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py", line 11
    Time Complexity: O(1) average
    ~~~~~
SyntaxError: invalid syntax
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter username: admin
Access Granted
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Instead of searching through a list one by one, we used a set. Searching in a set is much faster. This improves performance, especially when the data becomes large.

TASK-7

PROMPT:

Refactor into modular functions `add_book`, `remove_book`, and `search_book` with docstrings.

Your task is to refactor the code into a proper format

1. Create a module `library.py` with functions:
 - `add_book(title, author, isbn)`
 - `remove_book(isbn)`
 - `search_book(isbn)`
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

Library Management System (Unstructured Version)

This code needs refactoring into a proper module with documentation.

```
library_db = {}
```

Adding first book

```
title = "Python Basics"
```

```
author = "John Doe"
```

```
isbn = "101"
```

```
if isbn not in library_db:
```

```
    library_db[isbn] = {"title": title, "author": author}
```

```
    print("Book added successfully.")
```

```
else:
```

```
    print("Book already exists.")
```

```
# Adding second book (duplicate logic)
```

```
title = "AI Fundamentals"
```

```
author = "Jane Smith"
```

```
isbn = "102"
```

```
if isbn not in library_db:
```

```
    library_db[isbn] = {"title": title, "author": author}
```

```
    print("Book added successfully.")
```

```
else:
```

```
    print("Book already exists.")
```

```
# Searching book (repeated logic structure)
```

```
isbn = "101"
```

```
if isbn in library_db:
```

```
    print("Book Found:", library_db[isbn])
```

```
else:
```

```
    print("Book not found.")
```

```
# Removing book (again repeated pattern)
```

```
isbn = "101"
```

```
if isbn in library_db:
```

```
    del library_db[isbn]
```

```
    print("Book removed successfully.")
```

```
else:
```



```
print("Book not found.")
```

```
# Searching again
```

```
isbn = "101"
```

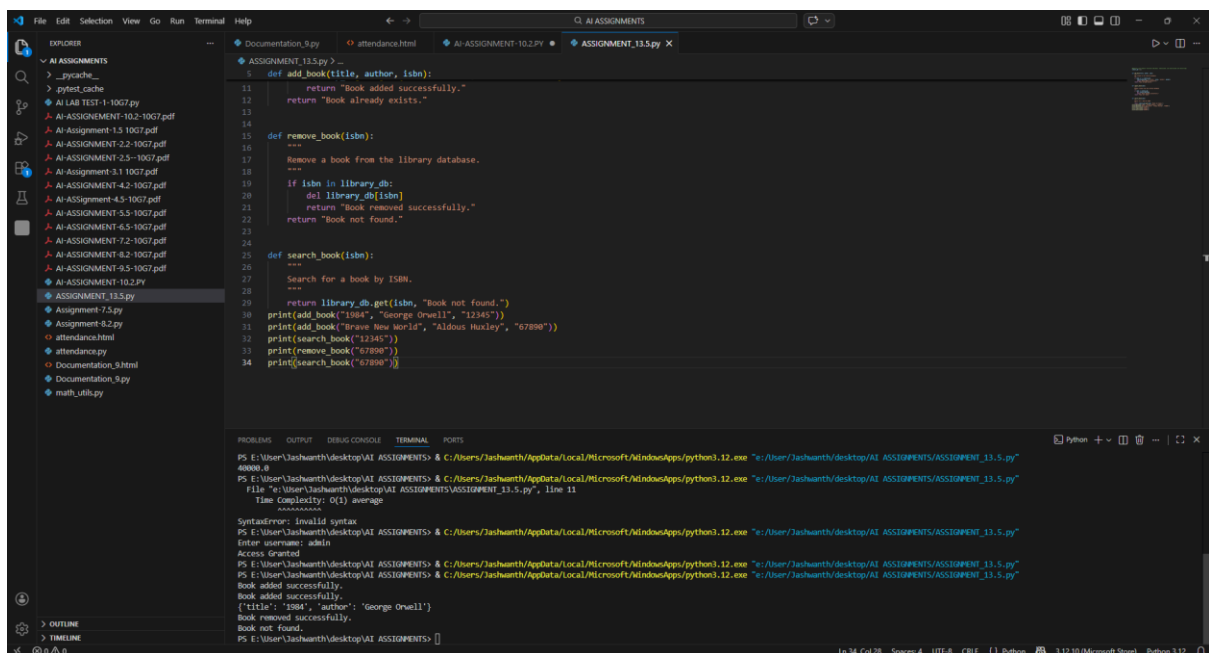
```
if isbn in library_db:
```

```
    print("Book Found:", library_db[isbn])
```

```
else:
```

```
    print("Book not found.")
```

CODE:



```
def add_book(title, author, isbn):
    return "Book added successfully."
    return "Book already exists."

def remove_book(isbn):
    """
    Remove a book from the library database.
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    return "Book not found."

def search_book(isbn):
    """
    Search for a book by ISBN.
    """
    return library_db.get(isbn, "Book not found.")

print(add_book("1984", "George Orwell", "12345"))
print(add_book("Brave New World", "Aldous Huxley", "67890"))
print(search_book("12345"))
print(remove_book("67890"))
print(search_book("67890"))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
40000.0
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
File "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py", line 11
    Time Complexity: O(1) average
    ~~~~~
SyntaxError: invalid syntax
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter isbn: 101
Access Granted
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Book added successfully.
{'title': '1984', 'author': 'George Orwell'}
Book removed successfully.
Book not found.
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

Library operations like adding, removing, and searching books were separated into different functions. This makes the system more organized and easier to maintain. Each function handles only one task.

TASK-8

PROMPT:

Create reusable function generate_fibonacci(n) with proper validation and docstrings.

Bad Code:

```
n=int(input("Enter limit: "))
```

```
a=0
```

```
b=1
```

```
print(a)
```

```
print(b)
```

```
for i in range(2,n):
```

```
c=a+b
```

```
print(c)
```

```
a=b
```

```
b=c"""
```

CODE:

```
1 #Create reusable function generate_fibonacci(n) with proper validation and docstrings.
2 def generate_fibonacci(n):
3     """
4     Generate Fibonacci sequence up to n terms.
5     """
6     if n <= 0:
7         return []
8     sequence = [0, 1]
9     for _ in range(2, n):
10         sequence.append(sequence[-1] + sequence[-2])
11     return sequence[:n]
12
13 print(generate_fibonacci(10))
```

OBSERVATION:

The Fibonacci logic was written inside a reusable function. It also checks for invalid input. This makes the code safe and reusable in other programs.

TASK-9

PROMPT:

Create functions `is_prime(n)` and `is_twin_prime(p1, p2)` with optimized logic.

Bad Code:

```
a=11
```

```
b=13
```

```
fa=0
```

```
for i in range(2,a):
```

```
if a%i==0:
```

```
fa=1
```

```
fb=0
```

```
for i in range(2,b):
```

```
if b%i==0:
```

```
fb=1
```

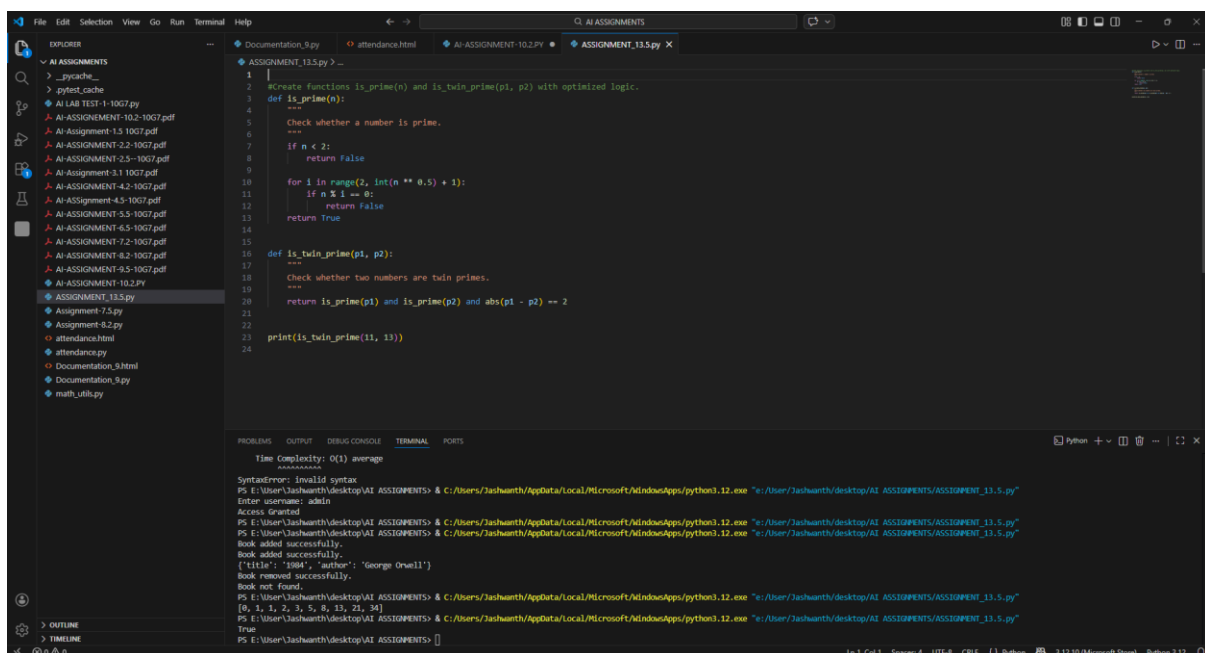
```
if fa==0 and fb==0 and abs(a-b)==2:
```

```
print("Twin Primes")
```

```
else:
```

```
print("Not Twin Primes")""
```

CODE:



```
1
2 #Create functions is_prime(n) and is_twin_prime(p1, p2) with optimized logic.
3 def is_prime(n):
4     """
5     Check whether a number is prime.
6     """
7     if n < 2:
8         return False
9
10    for i in range(2, int(n ** 0.5) + 1):
11        if n % i == 0:
12            return False
13    return True
14
15 def is_twin_prime(p1, p2):
16    """
17    Check whether two numbers are twin primes.
18    """
19    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
20
21
22 print(is_twin_prime(11, 13))
23
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Time Complexity: O(1) average

SyntaxError: Invalid syntax
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter username: admin
Access Granted
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Book added successfully.
['title': '1984', 'author': 'George Orwell']
Book removed successfully.
Book not found.
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[0, 1, 2, 3, 5, 8, 12, 21, 34]
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
True
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>

OBSERVATION:

The prime-checking logic was improved by checking only up to the square root of the number. This reduces unnecessary checks and improves speed. The twin prime logic was separated into another function for better clarity.

TASK-10

PROMPT:

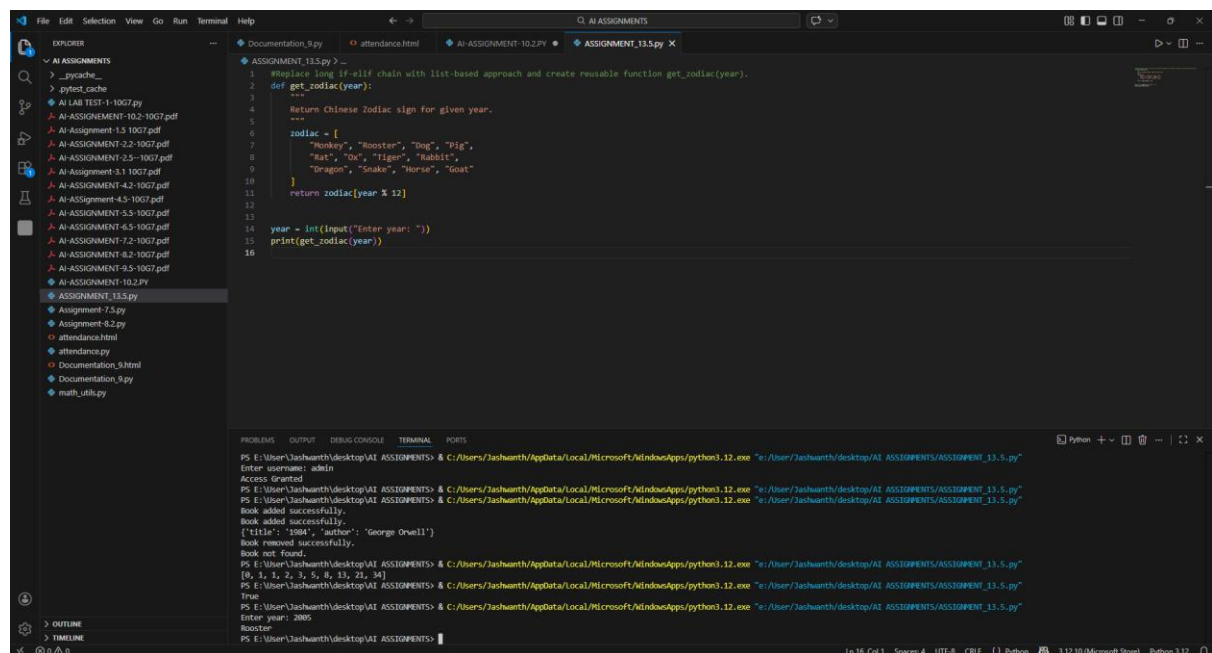
Replace long if-elif chain with list-based approach and create reusable function `get_zodiac(year)`.

Legacy Code:

```
year = int(input("Enter a year: "))

if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")"""
```

CODE:



```
1 #Replace long if-elif chain with list-based approach and create reusable function get_zodiac(year).
2 def get_zodiac(year):
3     """
4     Return Chinese Zodiac sign for given year.
5     """
6     zodiac = [
7         "Monkey", "Rooster", "Dog", "Pig",
8         "Rat", "Ox", "Tiger", "Rabbit",
9         "Dragon", "Snake", "Horse", "Goat"
10    ]
11    return zodiac[(year % 12)]
12
13
14 year = int(input("Enter year: "))
15 print(get_zodiac(year))
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"

Enter username: admin

Access Granted

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"

Book added successfully.

Book removed successfully.

Book not found.

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"

[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"

True

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"

Enter year: 2007

Rooster

PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>

OBSERVATION:

Instead of using a long chain of if-elif statements, we used a list and modulus operation. This makes the code shorter, cleaner, and easier to understand.

TASK-11

PROMPT:

Create a function is_harshad(number) to check if a number is divisible by the sum of its digits. Return True or False with proper validation and docstrings.

Legacy Code:

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```

print("True")

else:

print("False")

else:

print("False")"""

```

CODE:

```

1 #Harshad Number
2 def is_harshad(number):
3     """
4     Check if number is a Harshad number.
5     """
6     if number <= 0:
7         return False
8
9     digit_sum = sum(int(d) for d in str(number))
10    return number % digit_sum == 0
11
12
13 num = int(input("Enter number: "))
14 print(is_harshad(num))

```

Terminal Output:

```

PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Book added successfully.
{"title": "1881", "author": "George Orwell"}
Book removed successfully.
Book not found.
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[0, 1, 4, 2, 9, 5, 8, 10, 21, 24]
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
True
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter year: 2005
Rooster
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter number: 5
True
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>

```

OBSERVATION:

The program calculates the sum of digits in a simple and readable way. It checks whether the number is divisible by its digit sum. The implementation is clean and efficient.

TASK-12

PROMPT:

Create an optimized function count_trailing_zeros(n) to count trailing zeros in n! without calculating the factorial directly. Include validation and docstrings.

Legacy Code:

```

n = int(input("Enter a number: "))

fact = 1

i = 1

while i <= n:

```

```

fact = fact * i

i = i + 1

count = 0

while fact % 10 == 0:

count = count + 1

fact = fact // 10

print("Trailing zeros:", count)"""

```

CODE:

```

1 #trailing zeros
2 def count_trailing_zeros(n):
3     """
4     Count trailing zeros in factorial using optimized method.
5     """
6     if n < 0:
7         return 0
8
9     count = 0
10    while n > 0:
11        n //= 5
12        count += n
13
14    return count
15
16 print(count_trailing_zeros(100))

```

```

Book added successfully.
Book added successfully.
{"title": "1984", "author": "George Orwell"}
Book removed successfully.
Book not found.
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[0, 1, 2, 3, 5, 8, 12, 21, 34]
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
True
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter year: 2005
Rooster
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
True
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
24
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>

```

OBSERVATION:

Instead of calculating the full factorial (which is slow for large numbers), we used a mathematical method to count multiples of 5. This makes the program much faster and efficient.

TASK-13

PROMPT:

Create a function `collatz_sequence(n)` to generate the Collatz sequence until 1. Raise an error for non-positive input and include docstrings.

- Test Cases to Design:
 - Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
 - Edge: $1 \rightarrow [1]$

- Negative: -5
- Large: 27 (well-known long sequence)
 - Requirement: Validate correctness with pytest.

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow$ next = $n / 2$.
 - If n is odd $\rightarrow$ next = $3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.
- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)

2 (even $\rightarrow 2/2 = 1$)

CODE:


```
1 #Collatz Sequence
2 def collatz_sequence(n):
3     """
4     Generate Collatz sequence until 1.
5     """
6     if n <= 0:
7         raise ValueError("Number must be positive.")
8
9     sequence = []
10    while n != 1:
11        sequence.append(n)
12        n = n // 2 if n % 2 == 0 else 3 * n + 1
13
14    sequence.append(1)
15    return sequence
16
17
18 print(collatz_sequence(6))
```

```
{'title': '1984', 'author': 'George Orwell'}
Book removed successfully.
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
True
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter year: 2005
Rooter
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
Enter number: 5
True
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
24
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[6, 3, 10, 5, 16, 8, 4, 2, 1]
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

The function checks whether the input is valid before processing. It generates the sequence step by step until it reaches 1. The structure is clear and reusable.

TASK-14

PROMPT:

Create a function `generate_lucas(n)` to return the first `n` terms of the Lucas sequence with proper validation and docstrings.

- Function: Generate Lucas sequence up to `n` terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:

- Normal: $5 \rightarrow [2, 1, 3, 4, 7]$
- Edge: $1 \rightarrow [2]$
- Negative: $-5 \rightarrow \text{Error}$

Large: 10 (last element = 76).

- Requirement: Validate correctness with pytest

CODE:

```
1 # Lucas Sequence
2 def generate_lucas(n):
3     """
4     Generate Lucas sequence up to n terms.
5     """
6     if n <= 0:
7         raise ValueError("n must be positive")
8
9     sequence = [2, 1]
10    for _ in range(2, n):
11        sequence.append(sequence[-1] + sequence[-2])
12
13    return sequence[:n]
14
15
16 print(generate_lucas(10))
```

```
PS E:\User\Jashwanth\Desktop\VAE ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
PS E:\User\Jashwanth\Desktop\VAE ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
True
Enter year: 1800
Rooster
PS E:\User\Jashwanth\Desktop\VAE ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
28
PS E:\User\Jashwanth\Desktop\VAE ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[6, 3, 10, 5, 15, 8, 4, 2, 1]
PS E:\User\Jashwanth\Desktop\VAE ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/ASSIGNMENT_13.5.py"
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
PS E:\User\Jashwanth\Desktop\VAE ASSIGNMENTS>
```

OBSERVATION:

This is similar to the Fibonacci sequence but with different starting numbers. The function is clean and properly validated, making it reliable.

TASK-15

PROMPT:

Create a function `count_vowels_consonants(text)` to count vowels and consonants in a string, ignoring non-alphabet characters. Return the result as a tuple.

Function: Count vowels and consonants in string.

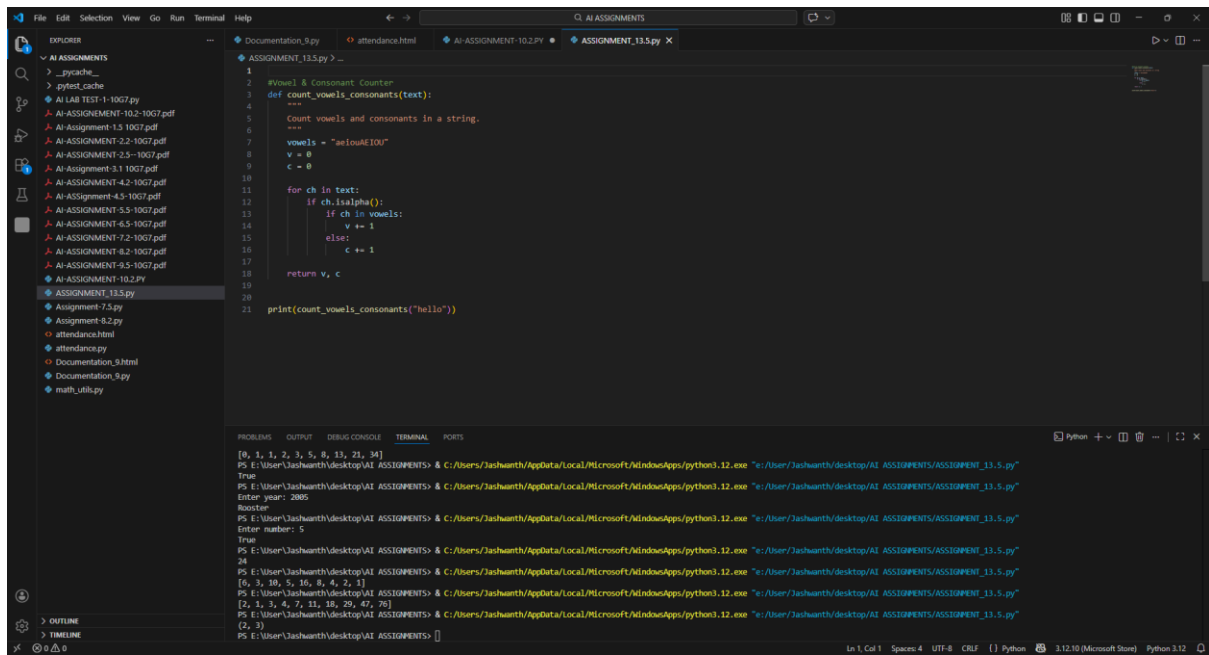
• Test Cases to Design:

- Normal: "hello" → (2,3)
- Edge: "" → (0,0)
- Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

CODE:



The screenshot shows a Visual Studio Code editor window with a Python file named `ASSIGNMENT_13.5.py` open. The file contains a function `count_vowels_consonants(text)` that counts vowels and consonants in a string. The function uses a set of vowels `"aeiouAEIOU"` and iterates through each character in the input string, incrementing counters for vowels and consonants. The function is called with the string `"hello"`, and the output is printed.

```
1 #Vowels & Consonant Counter
2 def count_vowels_consonants(text):
3     """
4     Count vowels and consonants in a string.
5     """
6     vowels = "aeiouAEIOU"
7     v = 0
8     c = 0
9
10    for ch in text:
11        if ch.isalpha():
12            if ch in vowels:
13                v += 1
14            else:
15                c += 1
16
17    return v, c
18
19
20
21 print(count_vowels_consonants("hello"))
```

The terminal output shows the execution of the script, including the function definition, the input string, and the resulting counts for vowels and consonants.

```
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
True
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
Enter year: 2085
Rooster
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
Enter number: 5
True
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
24
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
[6, 3, 18, 5, 16, 8, 4, 2, 1]
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
[2, 1, 3, 4, 7, 11, 18, 29, 47, 78]
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\ASSIGNMENT_13.5.py"
(2, 3)
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

The function counts only alphabet characters. It correctly separates vowels and consonants and ignores spaces or special characters. The logic is simple and accurate.